BridgeX Developer Manual — Core Edition

Illustrated engineering, operations, and security guide

作者: Manus AI

日期: 2026 年 08 月 19 日

目录

1 Purpose and operating boundary

BridgeX is a peer-to-peer global goods-carrying marketplace. It connects people who need an item carried with travelers or cargo-capacity providers who can legally and safely complete a compatible route. The engineering principle is **public discovery, protected execution**: route and service context can be public, but direct addresses, contact details, identity records, payment evidence, private payout instructions, and matched-deal chat are revealed only to the appropriate participant or authorized reviewer.

> This manual describes the present implementation and the safe path for future changes. It does not authorize developers or administrators to bypass row-level security, retrieve passwords, expose private records, or make legal, customs, banking, or carrier claims that BridgeX cannot substantiate.

1.1 Product claims that must remain accurate

The current manual-payment design is a **payment-proof review** workflow. A user supplies evidence for an administrator to review; that evidence should not be described as a bank balance, regulated escrow, cleared settlement, customs clearance, or legal approval. Members retain responsibility for accurate item information and for compliance with every relevant customs, airline, carrier, import/export, and destination-country requirement.

1.2 Roles and least privilege

Role	Core ability	Hard boundary
Guest	Browse public marketplace and profiles	Cannot post, respond, message, or view protected details
Member	Create posts, respond, manage own protected activity	Cannot view another member's documents, proof, or unrestricted messages
Administrator	Review assigned verification, payment, payout, report, and support queues	Cannot read or recover passwords; must use documented reason and audit trail
Super Admin	Manage governance and administrator controls	Does not override privacy or password rules

2 Architecture map

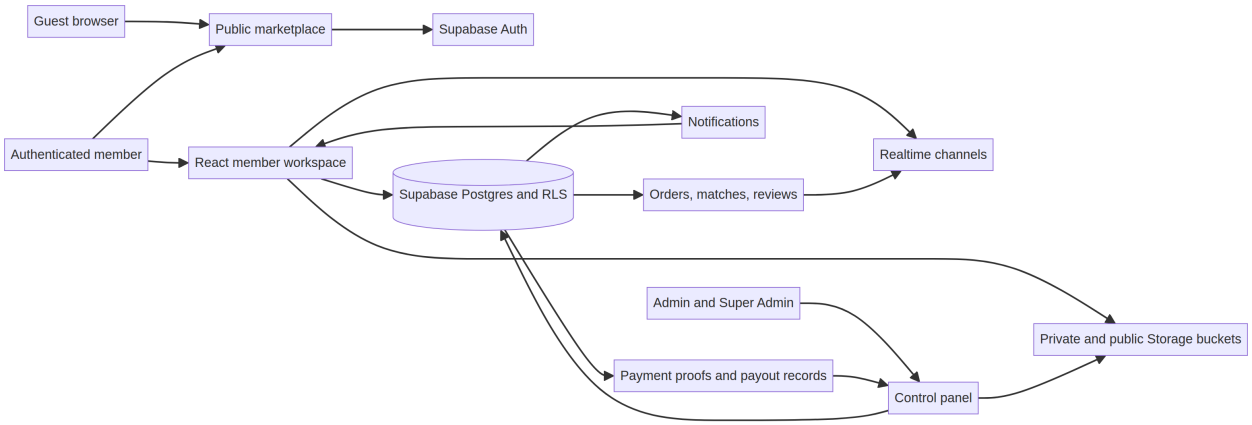

图 1 BridgeX architecture: public discovery, protected workspace, Supabase data controls, storage, realtime, and administrator operations.

2.1 Repository map

Location	Responsibility	Safe change pattern
`apps/web/client/src/App.tsx`	Routes and guards	Register a route and preserve role protection
`components/bridgex/PublicLayout.tsx`	Signed-in header and notification routing	Classify events by destination and preserve unread state
`pages/Workspace.tsx`	Member orders, offers, interests, release actions	Use in-app dialogs and explicit state labels
`pages/PaymentHistory.tsx`	Proof, payment history, traveler payout instructions	Never expose private payment data in public surfaces
`pages/AdminControl.tsx`	Administrative work queues	Add scoped search, decision reasons, and audit-oriented actions
`supabase/migrations/*.sql`	Schema, RLS, RPCs, triggers, buckets	Add forward-only migrations and commit them with the source change
`server/*.test.ts`	Business-rule regression coverage	Add a focused test when a workflow contract changes

2.2 Data classification

Public post media may be public only while linked to an open public post. Identity documents, payment proof screenshots, private payment instructions, and traveler payout assets require private buckets and database-backed authorization before a signed URL is created. A path string alone must never grant access.

3 Protected workflow state machines

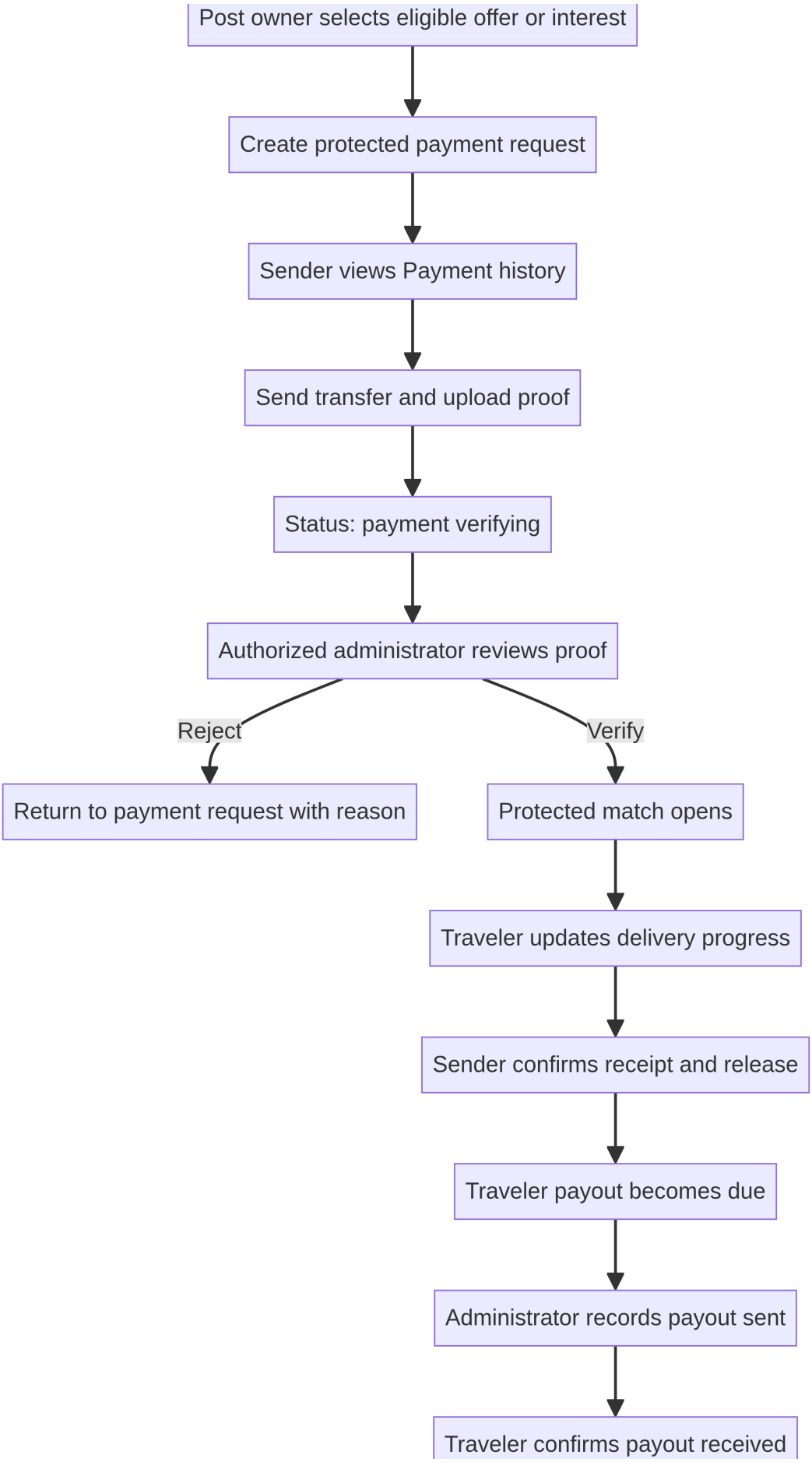
Post owner selects eligible offer or interest
Create protected payment request
Sender views Payment history
Send transfer and upload proof
Status: payment verifying
Authorized administrator reviews proof
Reject
Verify
Return to payment request with reason
Protected match opens
Traveler updates delivery progress
Sender confirms receipt and release
Traveler payout becomes due
Administrator records payout sent
Traveler confirms payout received

图 2 Protected payment proof, match opening, delivery release, and traveler payout sequence.

3.1 Sender request to traveler offer

A sender publishes a request with a route, item details, categories, weight, timing, handling needs, destination data, and a legality declaration. A traveler submits an offer. The sender selects an eligible offer; this creates a protected payment request in `pending_payment`. After evidence is submitted, status becomes `payment_verifying`. Only an authorized verification decision can open the protected match and reveal the matched workflow details.

3.2 Carry listing to sender interest

A traveler publishes spare capacity with route, departure and estimated-delivery dates, accepted categories, transport method, capacity, and pricing. A sender submits an interest with category, quantity, weight, deadline, and destination data. When the traveler selects an eligible interest, the sender's payment request is created and capacity is reserved during review. Capacity becomes a confirmed commitment only after payment proof is verified. Multiple interests can coexist while remaining capacity permits.

3.3 Delivery release and traveler payout

A traveler must enter private payout instructions before marking an order delivered. After the sender confirms receipt and releases the order, a traveler-payout-due record and administrator notification are created. An administrator records payout sent, and the traveler confirms payout received. This creates a closed, traceable record rather than leaving payout status in informal chat.

3.4 Payment states

State	Meaning	Permitted next action
`pending_payment`	Protected acceptance exists and requires member evidence	Submit proof or apply a policy-approved cancellation
`payment_verifying`	Proof is waiting for authorized review	Verify or reject with documented reason
`verified`	Review succeeded and protected match is open	Manage the protected order
`payment_due`	A released order requires traveler payout review	Review payout instruction and execute external payout under policy
`payment_sent`	Administrator recorded the external payout as sent	Traveler confirms receipt
`payment_received`	Traveler confirmed the payout	Retain immutable completion history

4 Payment and state-transition troubleshooting

4.1 First diagnostic rule

When a status error appears, do not patch only the visible table. Trace the entire state transition: the response record, request/listing record, payment record, protected match/order record, notification record, and any trigger/RPC that writes state. A feature can fail even if one table accepts the state but its companion table does not.

Symptom	Likely cause	Safe resolution
`offers_status_check`	Offer constraint omitted the payment state	Create a migration extending only the required response values, then test the transition
`listing_interests_status_check`	Interest constraint is older than the payment workflow	Update the named constraint and dependent UI labels
`send_requests_status_check`	Request tries `payment_pending` but its lifecycle rule omitted it	Extend the request rule using an append-only migration
`carry_listings_status_check`	Carry listing tries `payment_pending` but its lifecycle rule omitted it	Extend only the listing lifecycle values required by payment review
Private QR does not load	Signed URL authorization or storage policy is missing/misaligned	Verify active payment record, role scope, signed URL path, and bucket policy
Match does not open	Verification RPC failed or completed only part of its transaction	Inspect proof decision, RPC error, match creation, and notification insert in order

4.2 Standard safe migration sequence

```
1. Inspect the live constraint and the current forward migration history.
2. Add one new timestamped migration; never edit an already-applied migration.
3. Update every typed status label, filter, queue, and notification recipient.
4. Add a regression test for the intended transition and its safety boundary.
5. Apply the migration, verify the live definition, then run type check, tests, and production build.
6. Commit the migration and source code together so a new environment can reproduce production.
```

5 Notifications, messages, and feedback

Each notification must contain a stable event type, recipient, title, body, link target, related record, creation time, and read state. The signed-in navigation separates Profile, Workspace, Messages, Payment history, and Administrator destinations. Temporary in-app popups show new updates once; the permanent state remains in the relevant inbox.

5.1 Payment notification routing

Event	Recipient	Permanent destination
Payment proof submitted	Authorized administrator	Payment verification queue and review record
Payment verified for a sender request	Sender only	Payment history, protected match, and message update
Payment verified for a carry-space interest	Traveler only	Payment history, protected match, and message update
Traveler payout due	Authorized administrator	Traveler payout queue and payout record
Payout sent	Traveler	Payment history and payout record
Payout received	Authorized administrator	Payout queue/history and audit event

5.2 Reviews and reports

A completed or released protected order is the eligibility boundary for ratings. The review component must permit a one-to-five star selection and an optional factual comment, prevent duplicate submissions, and display policy errors clearly. A rating is a community signal, not a guarantee. Keep `0.0 (0)` visible for a member with no completed-order reviews rather than implying an unearned positive rating.

6 Administrator operations

6.1 Verification review

Review a member record, not disconnected document rows. The review surface should show the submitted identity details, all required document types, states, and an explicit approval or rejection reason. A rejection should notify the member with a correctable explanation and must not reveal confidential internal detection criteria.

6.2 Payment proof and payout review

Payment reviewers confirm that a payment record exists, evidence is associated with the expected payer, the amount and currency context are consistent, and the proof is legible. If the case cannot be accepted, reject with a concise, documented reason. Do not release contact data manually: that action belongs to

the protected verifier transaction. Payout reviewers should similarly record the reason and reference for payment sent, then wait for traveler receipt confirmation.

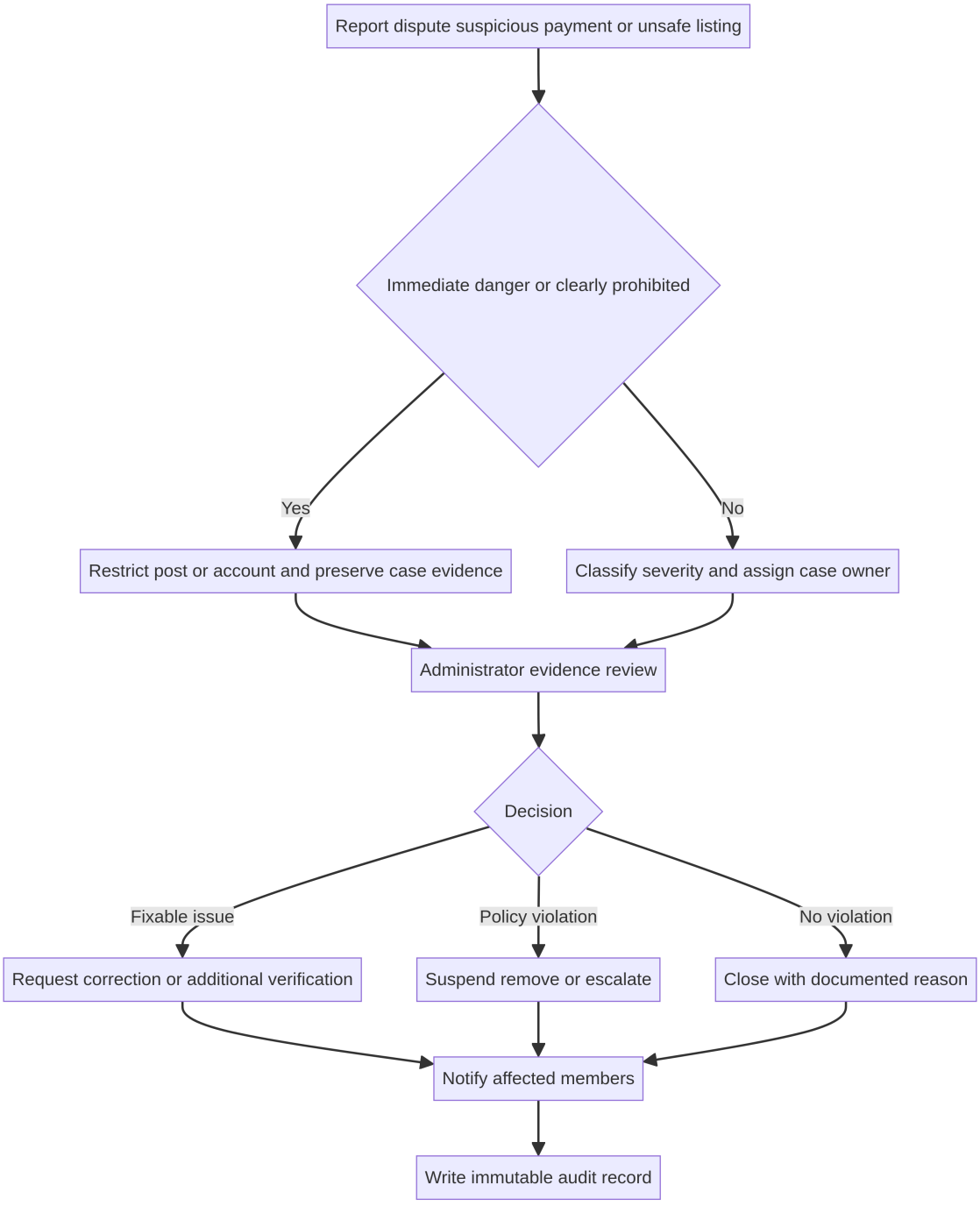

图 3 Incident response: classify, preserve evidence, decide under policy, notify, and create an audit record.

6.3 Incident response boundary

Reports, disputes, suspicious proof, and unsafe-listing claims must be classified by severity. Preserve relevant record references, restrict only within policy, and write a decision reason. Customer support should not make legal conclusions in chat. If an external report is required, record the authority, time, policy/legal basis, and material disclosed.

7 Release engineering and quality gates

7.1 Required change process

```
1. Record the user-visible work item in the `apps/web` implementation tracker.
2. Read the relevant UI page, tests, migration history, and data contract.
3. Make the smallest coherent implementation change.
4. Add or update a regression test.
5. Run pnpm check, pnpm test, and pnpm build inside apps/web.
6. Mark the tracker item complete only after validation.
7. Commit application source and matching migration together, then verify the
affected production path.
```

7.2 Security release gate

Before release, check role guards, RLS policy, storage policy, signed URL scope, notification recipient, error copy, client bundle exposure, test results, and production build. For changes involving documents, money, private addresses, or restrictions, add a second reviewer and retain decision context. Passwords must be reset through the authentication recovery flow; they cannot be displayed or reconstructed by any platform role.

8 Core audit findings

The current audit found a meaningful functional base: public marketplace, posts, offers/interests, protected matches, payment proof review, traveler payout history, messaging, ratings, reports, verification, and administrator governance. The production review showed guest marketplace posts after initial loading and showed in-app unread payment updates. The current repository contains 42 migrations, nine test files, and 55 passing automated checks.

The principal weaknesses are operational rather than cosmetic. Manual proof review is not automated settlement. The 5% service fee should evolve into a transaction-level fee ledger before it is charged at scale. Private document and payout records require retention and access-log policies. Remaining browser-native confirmation/alert calls should be replaced with accessible in-app dialogs. Data-quality validation needs to prevent incomplete route/city displays. Notification delivery needs event observability, and mobile release testing should be treated as a release gate.

9 600-page illustrated-manual publication plan

This core manual is the technical foundation, not a dishonest attempt to fill 600 pages with repeated text. The complete publication should be authored as eight image-rich volumes with screenshots, data-flow diagrams, policy examples, migration excerpts, test cases, and operations exercises.

Volume	Pages	Coverage
I. Product and domain foundation	60	User journeys, terminology, policy boundaries, route and item examples

Volume	Pages	Coverage
II. Frontend architecture	80	React routes, layouts, hooks, forms, accessibility, visual states, screenshots
III. Supabase data model	100	Tables, state diagrams, RLS reasoning, RPC contracts, storage buckets, migrations
IV. Payments and payouts	80	Proof review, fee ledger, payout due, reconciliation, disputes, audit records
V. Messages, notifications, and reviews	60	Conversation permissions, realtime, unread states, feedback eligibility, reports
VI. Administration and support	60	Verification, payment queues, decision templates, escalation runbooks
VII. Mobile, deployment, and observability	60	Android, Render, domains, monitoring, release/rollback runbooks
VIII. Troubleshooting and security appendices	100	Error catalogue, incident response, retention, QA cases, glossary, marked images
Total planned publication	**600**	**Non-duplicative illustrated engineering manual series**

10 References

[1] Mokhberi et al., “Trust, Privacy, and Safety Factors Associated with Decision Making in P2P Markets,” ACM CHI 2024, https://dl.acm.org/doi/full/10.1145/3613904.3641966.

[2] Persona, “Marketplace Trust & Safety Workshop,” https://www.youtube.com/watch?v=sh0meBwTdhI.

[3] General Administration of Customs of the People’s Republic of China, “Customs Clearance Guide for International Passengers,” http://english.customs.gov.cn/statics/88707c1e-aa4e-40ca-a968-bdbdbb565e4f.html.

[4] State Council of the People’s Republic of China, “Customs FAQ for Inbound and Outbound Passengers,” https://english.www.gov.cn/services/visitchina/202008/04/content_WS5f2905dec6d029c1c26372f0.html.